

PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Aplicaciones en Finanzas con Machine Learning | Segundo semestre 2026

Tarea 2

Detección de fraude en reclamos de seguros

Integrantes:

Fernán Gazmuri

Vicente Riffo

Albert Roca

Profesor: Joaquín Aguirre

Índice

1. Introducción y motivación **2**

2. Datos y preparación de la muestra **2**
2.1. Preparación y revisión de los datos 2
2.2. Análisis exploratorio de las variables 4

3. Metodología y modelos utilizados **4**
3.1. K vecinos más cercanos (KNN) 5
3.2. Regresión logística 5
3.3. Random Forest 5
3.4. XGBoost 6

4. Resultados **7**
4.1. Comparación del desempeño fuera de muestra 7
4.2. Resultados de Random Forest y efecto del umbral 8
4.3. Resultados de XGBoost e importancia de las variables 9

5. Conclusiones **9**

6. Reproducibilidad del análisis **10**

7. Declaración de uso de inteligencia artificial **11**

Anexo de figuras **12**

1. Introducción y motivación

La detección de fraude en reclamos de seguros es un problema relevante para las aseguradoras, ya que resulta difícil distinguir entre un siniestro legítimo y uno que ha sido inventado o cuyo importe ha sido exagerado. Esta dificultad se explica, en parte, por la asimetría de información entre el asegurado y la compañía: el primero dispone de información sobre las circunstancias del incidente que la aseguradora no siempre puede comprobar directamente.

Investigar todos los reclamos supone un costo importante y puede retrasar el pago de indemnizaciones legítimas. Por otro lado, aprobar sin revisión suficientes reclamos fraudulentos aumenta el costo de los siniestros y perjudica la rentabilidad de la aseguradora. Por tanto, resulta interesante estudiar si es posible utilizar modelos de Machine Learning para identificar qué casos presentan una mayor probabilidad de fraude y priorizar su revisión.

En este trabajo comparamos cuatro modelos de clasificación: K vecinos más cercanos (KNN), regresión logística, Random Forest y XGBoost. Nuestra variable objetivo es `fraud_reported`, que toma el valor 1 cuando el siniestro aparece registrado como fraude y 0 en caso contrario. El objetivo es evaluar qué modelo consigue distinguir mejor ambas clases y analizar hasta qué punto sus predicciones podrían resultar útiles para una aseguradora.

Es importante señalar que una predicción positiva no demuestra que un reclamo sea fraudulento. La utilidad del modelo estaría en ayudar al Departamento de Siniestros o a la Unidad de Investigaciones Especiales a decidir qué expedientes conviene revisar con mayor atención, sin sustituir la investigación correspondiente.

Para el análisis suponemos que la predicción se realiza después de recibir la declaración del siniestro y registrar los importes reclamados y las principales características del incidente. Esta distinción es relevante, ya que utilizar información que solo se conoce después de investigar un caso podría producir resultados aparentemente buenos, pero imposibles de reproducir en una situación real.

El conjunto de datos utilizado procede de *Auto Insurance Claims Data*, disponible en Kaggle:

`https://www.kaggle.com/datasets/buntyshah/auto-insurance-claims-data`.

2. Datos y preparación de la muestra

El archivo utilizado, `insurance_claims_limpio.csv`, contiene 1.000 observaciones, 35 variables explicativas y una variable objetivo. De los 1.000 siniestros, 247 aparecen clasificados como fraude, lo que representa un 24,7 % de la muestra, mientras que los 753 restantes corresponden a casos sin fraude reportado.

Esta distribución indica que las clases no están equilibradas, aunque tampoco nos encontramos ante una clase positiva extremadamente infrecuente. Por ello, además de medir la capacidad general de clasificación, resulta importante analizar cuántos fraudes consigue detectar cada modelo y cuántos reclamos legítimos clasifica incorrectamente.

2.1. Preparación y revisión de los datos

Durante la preparación del conjunto de datos eliminamos el número de póliza, el código postal y la dirección del incidente, debido principalmente a su elevada cardinalidad. También se eliminó una columna residual vacía, denominada `_c39`.

A partir de las fechas de contratación de la póliza y del incidente construimos tres variables temporales: los días transcurridos desde la emisión de la póliza, el mes del incidente y el día de la

semana en que ocurrió. Posteriormente retiramos las fechas originales de la matriz de predictores y transformamos la variable objetivo de las categorías `Y/N` a valores binarios `1/0`.

Para comprobar la consistencia de los datos preparados, realizamos una revisión de las observaciones y contrastamos el archivo utilizado con una copia pública del conjunto de datos original. No encontramos filas duplicadas, columnas constantes ni espacios sobrantes en las categorías de texto. Sin embargo, sí identificamos algunos valores ausentes y observaciones que requieren una interpretación más cuidadosa.

Cuadro 1: Principales hallazgos de la revisión de datos

Variable o comprobación	Casos	Observación
`authorities_contacted` vacío	91	En el archivo original estos casos contenían la categoría `None`, que se convirtió en una celda vacía durante la preparación.
`collision_type = MISSING`	178	Corresponden a 94 robos de vehículo y 84 incidentes con automóviles estacionados. En estos casos, la ausencia parece indicar que la variable no aplica.
`property_damage = MISSING`	360	No es posible determinar únicamente con los datos si se trata de información desconocida, no registrada o no aplicable.
`police_report_available = MISSING`	343	Es necesario conocer el significado de esta categoría y en qué momento está disponible la información policial.
`umbrella_limit <0`	1	La observación `row_id=290` presenta un límite de cobertura de -1.000.000, valor que ya estaba en el archivo original.
`dias_desde_emision <0`	1	La observación `row_id=578` presenta -20 días porque la fecha de emisión de la póliza es posterior al incidente.

En total, 660 observaciones contienen al menos una celda vacía o la categoría literal `MISSING`. Esto no significa necesariamente que todas presenten errores, ya que algunas ausencias pueden tener una explicación relacionada con el tipo de siniestro. Por ejemplo, no tendría mucho sentido exigir un tipo de colisión en un caso de robo de vehículo.

El archivo original contiene 1.000 filas y 39 columnas. Al compararlo con el archivo preparado comprobamos que se conservaron todas las observaciones, se eliminaron seis columnas y se incorporaron tres variables temporales. También verificamos que estas últimas se calcularon correctamente a partir de las fechas originales en las 1.000 filas.

La variable `total_claim_amount`, que fue eliminada durante la preparación, corresponde exactamente a la suma de los tres componentes del importe reclamado. Asimismo, los valores `?` del archivo original fueron sustituidos por la categoría `MISSING`.

La principal diferencia no deseada que encontramos fue la pérdida de la categoría `None` en `authorities_contacted`. En el resto de los predictores conservados no identificamos diferencias respecto al archivo original. Las dos anomalías numéricas señaladas también estaban presentes en los datos de origen y ambas quedaron en la muestra de entrenamiento.

Decidimos mantener el archivo preparado sin modificaciones adicionales para que los cuatro modelos pudieran compararse utilizando exactamente la misma base de datos.

Por otro lado, identificamos 95 casos en los que `months_as_customer` supera el número de meses transcurridos desde que el asegurado cumplió 18 años. Esto podría parecer una inconsistencia, pero no necesariamente implica un error, ya que desconocemos si la variable recoge la antigüedad personal del asegurado, una relación familiar previa u otro tipo de vínculo con la compañía.

Además, la correlación entre `months_as_customer` y los días desde la emisión de la póliza es de solo 0,048. Esto sugiere que probablemente representan conceptos distintos, aunque sería necesario conocer la definición exacta de ambas variables.

2.2. Análisis exploratorio de las variables

Una de las cuestiones más llamativas del conjunto de datos aparece al estudiar la relación entre determinadas variables categóricas y la frecuencia de fraude reportado.

Mientras que la tasa de fraude global es del 24,7 %, entre los asegurados cuya afición aparece registrada como `chess` alcanza el 82,6 %, con 38 casos positivos de un total de 46. Para `cross-fit`, la proporción es del 74,3 %, con 26 positivos entre 35 observaciones.

Si agrupamos ambas categorías, encontramos 64 fraudes reportados entre 81 casos, equivalentes al 79,0 %. En el resto de las aficiones, la proporción se reduce al 19,9 %, con 183 positivos entre 919 observaciones.

También observamos diferencias importantes según la severidad del incidente. Entre los casos clasificados como `Major Damage`, 167 de 276 corresponden a fraude reportado, lo que representa un 60,5 %. En cambio, para `Total Loss`, la proporción es del 12,9 %, con 36 positivos entre 280 casos.

Estas diferencias indican que las aficiones y la severidad del incidente pueden contener información predictiva relevante. Sin embargo, la magnitud de las asociaciones resulta poco intuitiva desde un punto de vista económico. No podemos concluir que una determinada afición aumente la probabilidad real de cometer fraude, sino únicamente que existe una asociación muy fuerte dentro de esta muestra.

Para estudiar hasta qué punto el desempeño de Random Forest depende de estas variables, realizamos una comprobación exploratoria mediante validación cruzada de cinco pliegues sobre entrenamiento. Manteniendo fijos los hiperparámetros seleccionados, obtuvimos una AUC media de 0,857 utilizando todas las variables, 0,750 al excluir las aficiones, 0,714 al excluir la severidad y 0,600 al eliminar ambas.

Por tanto, una parte considerable de la capacidad predictiva parece concentrarse en estas dos características. Esta comprobación no constituye una comparación definitiva entre nuevas especificaciones, ya que no volvimos a optimizar los hiperparámetros después de eliminar variables. Aun así, pone de manifiesto la importancia de comprobar si estas relaciones también aparecen en otras muestras y si ambas variables estarían disponibles en el momento de realizar la predicción.

Finalmente, observamos que los incidentes están concentrados en un periodo temporal reducido: 516 corresponden a enero, 472 a febrero y únicamente 12 a marzo. Al recuperar las fechas del archivo original comprobamos que los siniestros se sitúan entre el 1 de enero y el 1 de marzo de 2015.

En este trabajo utilizamos una separación aleatoria estratificada para poder comparar los modelos bajo las mismas condiciones. No obstante, una extensión interesante sería realizar una validación temporal utilizando las fechas originales, lo que permitiría comprobar si el desempeño se mantiene al pronosticar incidentes posteriores.

3. Metodología y modelos utilizados

Para comparar los diferentes algoritmos dividimos la muestra en un 80 % de entrenamiento y un 20 % de prueba, manteniendo la proporción de ambas clases mediante una partición estratificada y utilizando la semilla 42.

De esta forma, contamos con 800 observaciones para entrenar y seleccionar los modelos y 200 observaciones para evaluar su capacidad predictiva fuera de muestra. Entre estas últimas hay 49 casos positivos.

La asignación de cada observación se guardó en el archivo `resultados_random_forest/particion.csv`, utilizando el identificador `row_id`. Los cuatro modelos emplean esta misma partición, evitando que las diferencias de resultados puedan atribuirse a haber utilizado muestras de prueba distintas.

Además, los hiperparámetros se seleccionaron utilizando únicamente información de entrenamiento. La muestra de prueba quedó reservada para la evaluación final.

3.1. K vecinos más cercanos (KNN)

El primer modelo utilizado es K vecinos más cercanos, que clasifica cada observación a partir de las clases de los registros más similares presentes en entrenamiento.

Para aplicar este algoritmo tuvimos que transformar las variables categóricas mediante codificación *one-hot*, que crea variables binarias para representar las distintas categorías. Posteriormente estandarizamos las variables, ya que KNN utiliza distancias y, sin escalado, las variables con valores numéricos más grandes podrían dominar el cálculo de similitud.

Tanto la codificación como el escalado se ajustaron dentro de cada pliegue de validación, evitando utilizar información que no correspondía a la muestra de entrenamiento de ese pliegue. Los valores vacíos de la variable de autoridades se trataron como una categoría separada.

Para seleccionar el modelo comparamos 28 combinaciones de número de vecinos, sistema de ponderación y métrica de distancia, utilizando validación cruzada estratificada de cinco pliegues.

La combinación con mayor F1 medio fue la de tres vecinos, distancia euclidiana y ponderación uniforme. El F1 medio de validación fue de 0,2571. Una vez seleccionados los hiperparámetros, utilizamos el voto mayoritario de los vecinos para determinar la clase predicha.

La búsqueda de hiperparámetros y el ajuste final requirieron aproximadamente 5,3 segundos en la ejecución realizada.

3.2. Regresión logística

Como segundo modelo utilizamos una regresión logística, que permite estimar la probabilidad de pertenecer a una clase a partir de una combinación lineal de las variables explicativas.

A diferencia de KNN, este modelo no basa la clasificación en la distancia entre observaciones, sino que relaciona las variables con el logaritmo del cociente entre la probabilidad de fraude y la probabilidad de no fraude.

Aplicamos la misma codificación de variables categóricas y el mismo escalado utilizados en KNN, ajustados con los datos de entrenamiento.

Para considerar la diferencia en frecuencia entre ambas clases utilizamos `class_weight=balanced`, que asigna pesos inversamente proporcionales a su frecuencia. También fijamos un máximo de 1.000 iteraciones y un umbral de clasificación de 0,5.

Incluimos la regresión logística como modelo de referencia porque permite evaluar si los algoritmos más complejos consiguen mejorar de forma relevante una especificación relativamente sencilla e interpretable.

El ajuste requirió aproximadamente 0,04 segundos en la ejecución realizada.

3.3. Random Forest

El tercer modelo es Random Forest, un algoritmo que combina las predicciones de múltiples árboles de decisión. Cada árbol se estima utilizando variaciones de los datos de entrenamiento y considera

subconjuntos aleatorios de variables en sus divisiones. Esto permite reducir parte de la inestabilidad que tendría un único árbol y capturar relaciones no lineales e interacciones entre variables.

Para preparar los datos construimos un *pipeline* en el que las variables numéricas con valores ausentes se imputan mediante la mediana, mientras que las categóricas se imputan utilizando la moda. Posteriormente aplicamos codificación *one-hot* a las variables categóricas, permitiendo también la presencia de categorías nuevas en prueba.

La imputación y la codificación se estimaron únicamente con los datos de entrenamiento y dentro de cada pliegue de validación. Con ello evitamos que el preprocesamiento incorporara información de las observaciones utilizadas para evaluar el modelo.

Las categorías literales `MISSING` se mantuvieron como categorías independientes. Sin embargo, los vacíos de `authorities_contacted` fueron imputados mediante la moda. Como comprobamos posteriormente, estas observaciones correspondían originalmente a la categoría `None`, por lo que se produjo una pérdida de información.

Esta diferencia debe tenerse en cuenta al interpretar los resultados. En una nueva versión del análisis sería conveniente recuperar la categoría original y volver a entrenar todos los modelos con el mismo conjunto de datos corregido.

Para la estimación utilizamos un bosque de 150 árboles y `max_features=sqrt`. Comparamos ocho configuraciones combinando dos valores de profundidad máxima (sin límite y 8), dos valores del mínimo de observaciones por hoja (1 y 5) y dos opciones de ponderación de clases (sin ajuste y `balanced_subsample`).

La selección se realizó mediante validación cruzada estratificada de cinco pliegues, utilizando la AUC media como criterio. La configuración elegida fue una profundidad máxima de 8, mínimo de una observación por hoja y ausencia de ponderación adicional de clases.

Este modelo obtuvo una AUC media de validación de 0,8574, con una desviación estándar de 0,0286 entre pliegues. La búsqueda de hiperparámetros tomó 65,84 segundos en el equipo utilizado, aunque este tiempo depende del hardware.

Posteriormente seleccionamos el umbral de clasificación utilizando predicciones fuera de pliegue de las 800 observaciones de entrenamiento. En concreto, escogimos el umbral que maximizaba F1, obteniendo un valor de 0,309.

Este ajuste es importante porque el umbral convencional de 0,5 no tiene por qué ser el más adecuado en un problema donde interesa detectar un número elevado de fraudes. Para analizar su efecto, también presentamos los resultados que se obtendrían manteniendo el umbral de 0,5.

3.4. XGBoost

Finalmente, utilizamos XGBoost, un algoritmo de *gradient boosting* que construye árboles de decisión de forma secuencial. A diferencia de Random Forest, donde los árboles se combinan principalmente para reducir la variabilidad, XGBoost va incorporando nuevos árboles para corregir los errores del conjunto anterior.

Para las variables categóricas aplicamos codificación *one-hot* dentro de cada pliegue de validación, manteniendo los valores vacíos de autoridades como una categoría independiente.

La selección de hiperparámetros se realizó mediante una búsqueda aleatoria de 200 configuraciones, utilizando validación cruzada estratificada de cinco pliegues y la AUC media como criterio de selección.

En la ejecución realizada con XGBoost 3.2.0, la configuración elegida utilizó 300 árboles, profundidad máxima de 6, tasa de aprendizaje de 0,05 y peso 1 para la clase positiva.

La AUC media de validación fue de 0,866, ligeramente superior a la obtenida con Random Forest.

Al igual que en Random Forest, seleccionamos el umbral maximizando el F1 a partir de predicciones fuera de pliegue del conjunto de entrenamiento. El umbral elegido fue de 0,26.

La búsqueda de hiperparámetros tomó aproximadamente 82,41 segundos y el ajuste final 0,22 segundos en el equipo utilizado.

4. Resultados

Una vez seleccionados los modelos, evaluamos su desempeño utilizando AUC, precisión, recall y F1.

La AUC mide la capacidad del modelo para ordenar los casos positivos por encima de los negativos a partir de sus probabilidades estimadas. Por otro lado, la precisión indica qué proporción de los casos clasificados como fraude realmente pertenecen a esa categoría, mientras que el recall mide qué proporción de los fraudes existentes consigue detectar el modelo.

El F1 combina precisión y recall mediante su media armónica. Esta métrica resulta especialmente útil en nuestro problema, ya que buscamos detectar fraudes sin generar una cantidad excesiva de falsos positivos.

Cuadro 2: Desempeño de los modelos en entrenamiento y prueba

Modelo	Muestra	AUC	Precisión	Recall	F1
KNN	Entrenamiento	0,8630	0,7405	0,4899	0,5897
	Prueba	0,5003	0,2667	0,1633	0,2025
Regresión logística	Entrenamiento	0,9641	0,7480	0,9444	0,8348
	Prueba	0,7806	0,5439	0,6327	0,5849
Random Forest	Entrenamiento	0,9917	0,7085	0,9697	0,8188
	Prueba	0,8566	0,6364	0,8571	0,7304
XGBoost	Entrenamiento	0,9801	0,6679	0,8939	0,7646
	Prueba	0,8227	0,6364	0,8571	0,7304

4.1. Comparación del desempeño fuera de muestra

Los resultados muestran diferencias importantes entre los cuatro algoritmos, especialmente al comparar su desempeño en entrenamiento y prueba.

KNN es el modelo con peores resultados fuera de muestra. Aunque alcanza una AUC de 0,8630 en entrenamiento, esta cae hasta 0,5003 en prueba, un valor prácticamente equivalente al de una clasificación aleatoria. Además, únicamente consigue detectar 8 de los 49 fraudes reportados, con una precisión del 26,7 %, apenas superior a la prevalencia del 24,5 % en la muestra de prueba.

Una posible explicación es que, al utilizar numerosas variables numéricas y categóricas codificadas, la distancia entre observaciones deja de ser suficientemente informativa para identificar vecinos realmente similares. Por tanto, el buen ajuste de entrenamiento no se traduce en una capacidad predictiva útil sobre observaciones nuevas.

La regresión logística presenta una mejora considerable respecto a KNN. Su AUC en prueba alcanza 0,7806 y consigue detectar 31 de los 49 fraudes. Aunque su desempeño es inferior al de

los modelos basados en árboles, constituye una referencia interesante porque logra resultados razonables utilizando una estructura más sencilla.

Los mejores resultados corresponden a Random Forest y XGBoost. Random Forest alcanza una AUC de 0,8566 en prueba, frente a 0,8227 de XGBoost. Ambos modelos obtienen la misma precisión, recall y F1, con valores de 0,6364, 0,8571 y 0,7304, respectivamente.

Para analizar el posible sobreajuste también comparamos la AUC de entrenamiento y prueba. Las diferencias son aproximadamente 0,363 para KNN, 0,184 para la regresión logística, 0,135 para Random Forest y 0,157 para XGBoost.

Si realizamos la misma comparación utilizando F1, las diferencias son 0,387, 0,250, 0,088 y 0,034, respectivamente. Hay que tener en cuenta que las métricas dependientes del umbral corresponden a las reglas de clasificación elegidas para cada modelo, mientras que la AUC permite comparar directamente la capacidad de ordenamiento sin depender del umbral.

En conjunto, Random Forest presenta la mayor AUC en la muestra de prueba, aunque XGBoost consigue las mismas decisiones binarias y un F1 idéntico.

4.2. Resultados de Random Forest y efecto del umbral

Uno de los resultados más relevantes del trabajo aparece al estudiar cómo cambia el desempeño de Random Forest según el umbral utilizado para clasificar una observación como fraude.

Con el umbral de 0,309, seleccionado a partir de entrenamiento, la matriz de confusión contiene 127 verdaderos negativos, 24 falsos positivos, 7 falsos negativos y 42 verdaderos positivos.

Esto significa que, de los 200 siniestros de prueba, el modelo clasifica 66 como posibles fraudes. Entre ellos se encuentran 42 de los 49 casos que realmente aparecen registrados como fraude.

Desde el punto de vista de una aseguradora, este resultado podría interpretarse como la necesidad de revisar aproximadamente un 33 % de los siniestros para detectar el 85,7 % de los fraudes reportados.

Sin embargo, el resultado cambia considerablemente si mantenemos el umbral convencional de 0,5. En ese caso, el modelo detecta únicamente 4 de los 49 fraudes, con una precisión de 0,364, un recall de 0,082 y un F1 de 0,133.

Por tanto, la selección del umbral tiene un efecto muy importante sobre la utilidad práctica del modelo. Aunque las probabilidades estimadas sean exactamente las mismas, modificar el punto a partir del cual clasificamos un caso como fraude cambia el número de expedientes que se revisarían.

La AUC no se modifica al cambiar el umbral, ya que evalúa el ordenamiento de las probabilidades estimadas y no las decisiones binarias que se obtienen a partir de ellas.

Por otro lado, Random Forest presenta una AUC de entrenamiento de 0,9917, superior a la de prueba. Esta diferencia indica que el modelo se ajusta mejor a las observaciones utilizadas para entrenarlo, aunque la AUC media obtenida durante la validación cruzada, 0,8574, es muy próxima a la AUC final de prueba, 0,8566.

Esto sugiere que el proceso de validación consiguió aproximar razonablemente el desempeño dentro de esta muestra. No obstante, el tamaño reducido del conjunto de datos y la falta de una evaluación en periodos posteriores impiden asegurar que se mantendrían estos resultados en una aseguradora real.

Al analizar la importancia de las variables, encontramos que algunas de las características más relevantes son la presencia de daño grave, las aficiones `chess` y `cross-fit`, y el importe reclamado

por el vehículo.

Estas importancias se calcularon a partir de la reducción de impureza de los árboles. Por tanto, indican qué variables utiliza el modelo para realizar sus divisiones, pero no deben interpretarse como efectos causales. Además, esta medida puede favorecer determinadas variables en función de sus características.

4.3. Resultados de XGBoost e importancia de las variables

XGBoost también presenta un desempeño elevado fuera de muestra, con una AUC de 0,8227 y un F1 de 0,7304.

Aunque la AUC es inferior a la de Random Forest, ambos modelos detectan exactamente los mismos 42 fraudes en la muestra de prueba y generan 24 falsos positivos.

Al estudiar la importancia de las variables en XGBoost observamos que la severidad del incidente y las aficiones del asegurado concentran cerca del 60 % de la ganancia total del modelo.

Para comprobar la relevancia de estas variables también analizamos cómo cambia la AUC cuando alteramos sus valores mediante permutación en la muestra de prueba. Al permutar la severidad, la AUC disminuye aproximadamente 0,273, mientras que al permutar las aficiones cae 0,090.

Estos resultados refuerzan la conclusión obtenida durante el análisis exploratorio: una parte considerable de la capacidad predictiva del conjunto de datos está concentrada en unas pocas variables categóricas.

También estudiamos la sensibilidad del modelo a la profundidad máxima de los árboles. Al aumentar la profundidad de 1 a 10, la AUC de entrenamiento pasa de 0,933 a 0,982. Sin embargo, la AUC media de validación permanece aproximadamente entre 0,859 y 0,869.

Esto indica que aumentar la complejidad permite mejorar el ajuste sobre entrenamiento, pero no produce una mejora equivalente en la capacidad predictiva fuera de muestra. Por tanto, utilizar árboles más profundos no parece aportar una ventaja importante en este problema.

Finalmente, realizamos una comprobación exploratoria adicional utilizando una regresión logística que incluía únicamente la severidad del incidente y las aficiones del asegurado. Esta especificación obtuvo una AUC de 0,8444 en prueba.

Lo más llamativo es que esta regresión logística simplificada, Random Forest y XGBoost clasifican exactamente los mismos reclamos como positivos utilizando sus respectivos umbrales.

Estas decisiones también coinciden con una regla sencilla que identifica un reclamo cuando la severidad aparece como `Major Damage` o la afición del asegurado corresponde a `chess` o `cross-fit`.

Esta coincidencia ayuda a explicar por qué Random Forest y XGBoost obtienen el mismo F1, aunque sus probabilidades estimadas y sus valores de AUC sean diferentes.

Sin embargo, se trata de una comprobación exploratoria realizada después de analizar los resultados, por lo que no constituye evidencia suficiente de que una regla tan sencilla mantendría el mismo desempeño en otras muestras.

5. Conclusiones

Los resultados obtenidos muestran que los modelos basados en árboles presentan una mayor capacidad predictiva que KNN y la regresión logística para clasificar los reclamos de seguros del conjunto de datos utilizado.

En particular, Random Forest obtiene la mayor AUC en prueba, con un valor de 0,8566. Utilizando el umbral seleccionado a partir de entrenamiento, consigue detectar 42 de los 49 fraudes reportados, aunque para ello clasifica 66 de los 200 siniestros como casos que deberían revisarse.

XGBoost alcanza el mismo F1 y detecta exactamente los mismos casos positivos, pero presenta una AUC algo inferior. Por su parte, la regresión logística ofrece un resultado razonable con una estructura más sencilla, mientras que KNN muestra una capacidad de generalización muy limitada.

En base a estos resultados, seleccionaríamos Random Forest como modelo principal para ordenar los expedientes según su probabilidad estimada de fraude, ya que obtiene el mejor desempeño en términos de AUC fuera de muestra.

No obstante, la selección de un modelo para su utilización real no debería depender únicamente de estas métricas. También sería necesario conocer cuánto cuesta investigar un siniestro legítimo y cuál es la pérdida esperada cuando un fraude no se detecta.

En nuestro análisis seleccionamos el umbral maximizando F1, pero esto no significa necesariamente que sea el umbral económicamente óptimo para una aseguradora. Para determinarlo habría que incorporar los costos asociados a los falsos positivos y falsos negativos, además de comprobar si las probabilidades estimadas están correctamente calibradas.

Otra conclusión importante es la elevada dependencia de los modelos respecto a la severidad del incidente y las aficiones del asegurado. Estas variables permiten obtener buenos resultados dentro del conjunto de datos, pero algunas de sus asociaciones con el fraude son difíciles de justificar desde un punto de vista económico.

Por tanto, no podemos asumir que dichas relaciones se mantendrán en otras aseguradoras o en periodos posteriores. Antes de utilizar el modelo sería necesario revisar cómo se registraron estas variables, confirmar que están disponibles en el momento de realizar la predicción y evaluar su estabilidad fuera de la muestra analizada.

También convendría corregir la pérdida de la categoría `None` en la variable de autoridades y aclarar el significado de algunas observaciones anómalas. Si se realizan estas modificaciones, los cuatro modelos deberían volver a entrenarse y evaluarse utilizando el mismo protocolo, para mantener la comparabilidad de los resultados.

Como posible extensión futura, sería interesante realizar una validación temporal recuperando las fechas originales de los incidentes, además de evaluar los modelos con datos más recientes y de otras compañías.

En definitiva, los resultados muestran que Random Forest y XGBoost pueden ser útiles para identificar expedientes que merecen una revisión adicional. Sin embargo, la capacidad predictiva observada en este conjunto de datos no garantiza que los modelos mantengan el mismo desempeño en una aplicación real.

El principal desafío no es únicamente conseguir buenas métricas dentro de la muestra, sino comprobar que las relaciones utilizadas por los modelos son estables, que la información está disponible cuando se necesita y que las decisiones derivadas de sus predicciones resultan económicamente justificables.

6. Reproducibilidad del análisis

Para facilitar la reproducción de los resultados, el código utilizado se encuentra organizado en diferentes archivos correspondientes a los modelos y a la revisión de los datos.

El análisis de Random Forest puede reproducirse ejecutando `random_forest.py` sobre `insurance_claims_limpio.` utilizando las dependencias especificadas en `requirements.txt`.

Este archivo genera la partición común de entrenamiento y prueba, las métricas de evaluación, las predicciones, los resultados de validación cruzada, las importancias de las variables y la curva ROC. Los resultados se almacenan en la carpeta `resultados_random_forest/`.

Además, realizamos tres pruebas automáticas del flujo y comprobamos que las 200 predicciones de prueba son consistentes con las métricas presentadas en el informe.

La revisión descriptiva de los datos puede reproducirse mediante `auditoria_csv.py`, mientras que el notebook `modelos_knn_logistica.ipynb` contiene la implementación de KNN y la regresión logística.

En ambos casos, la codificación de variables categóricas y el escalado se realizan dentro del flujo de ajuste correspondiente. También conservamos el informe original y su notebook como material de referencia.

Por último, el archivo `xgboost_fraude.py` permite reproducir el análisis de XGBoost y la regresión logística exploratoria con dos variables. Las tablas, predicciones y gráficos correspondientes se guardan en `resultados_xgboost/`.

De esta forma, los resultados numéricos presentados en el informe pueden comprobarse ejecutando el código sobre el conjunto de datos utilizado.

7. Declaración de uso de inteligencia artificial

Durante el desarrollo de este trabajo utilizamos herramientas de inteligencia artificial generativa como apoyo en distintas etapas del análisis. En particular, las empleamos para asistirnos en tareas de programación, implementación, depuración y revisión del código utilizado para entrenar y evaluar los modelos de clasificación.

También recurrimos a estas herramientas durante la revisión del conjunto de datos, principalmente para apoyar la identificación de valores ausentes, posibles inconsistencias y diferencias entre el archivo preparado y el conjunto de datos original. Las utilizamos además para consultar y comprender algunos procedimientos relacionados con la validación de modelos, la selección de hiperparámetros y la interpretación de las métricas de clasificación.

El planteamiento del problema, las decisiones metodológicas, la selección y comparación de los modelos y la interpretación de los resultados fueron revisados por nosotros. Asimismo, las cifras y métricas presentadas en el informe proceden de la ejecución del código sobre el conjunto de datos utilizado y no de resultados numéricos generados directamente por herramientas de inteligencia artificial.

Asumimos la responsabilidad final sobre el contenido del trabajo, el funcionamiento del código, los cálculos realizados y las conclusiones presentadas.

Anexo de figuras

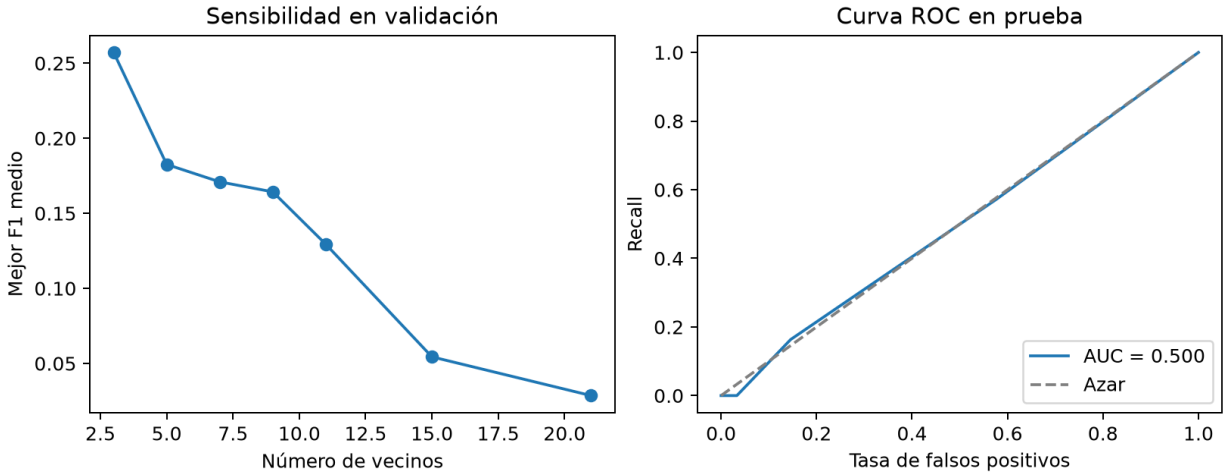


Figura 1: Sensibilidad de KNN en validación y curva ROC del modelo elegido en prueba.

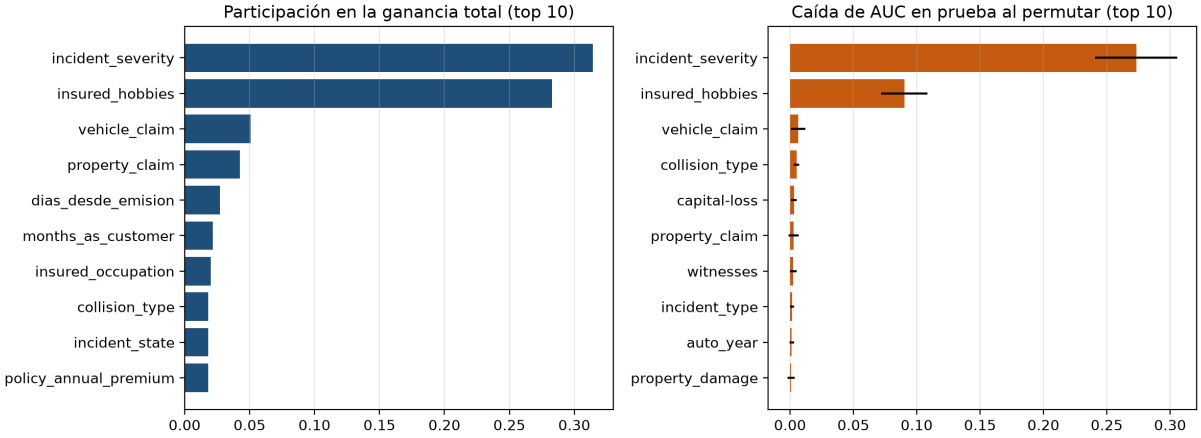


Figura 2: Importancia de variables en XGBoost por ganancia y por permutación.

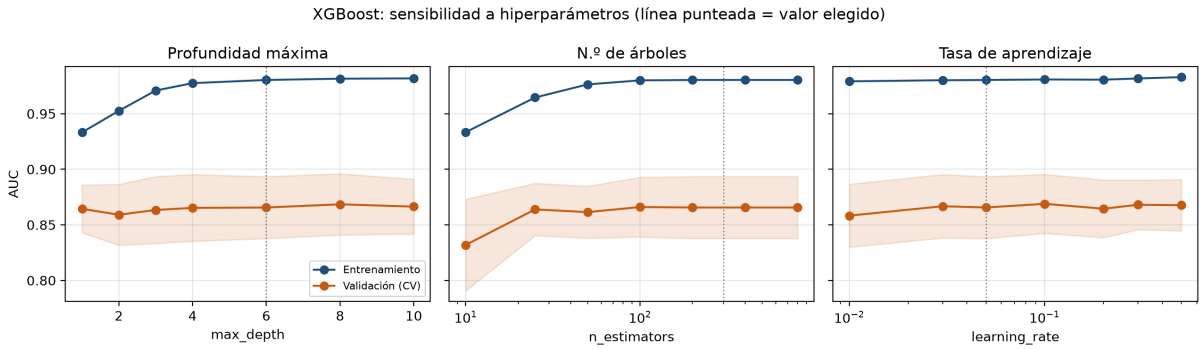


Figura 3: Sensibilidad de XGBoost a los hiperparámetros en validación cruzada.

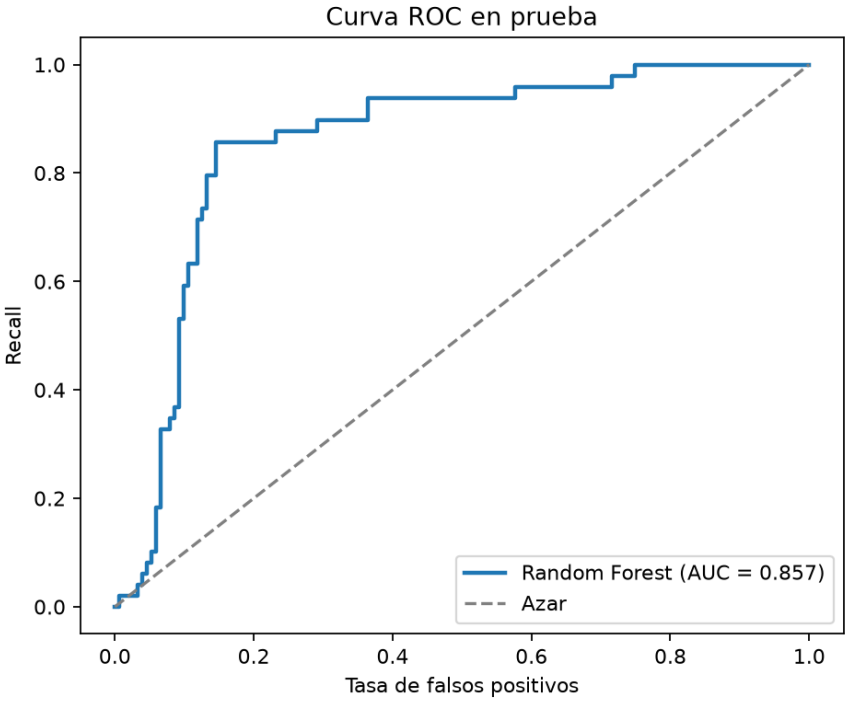


Figura 4: Curva ROC de Random Forest en la muestra de prueba.